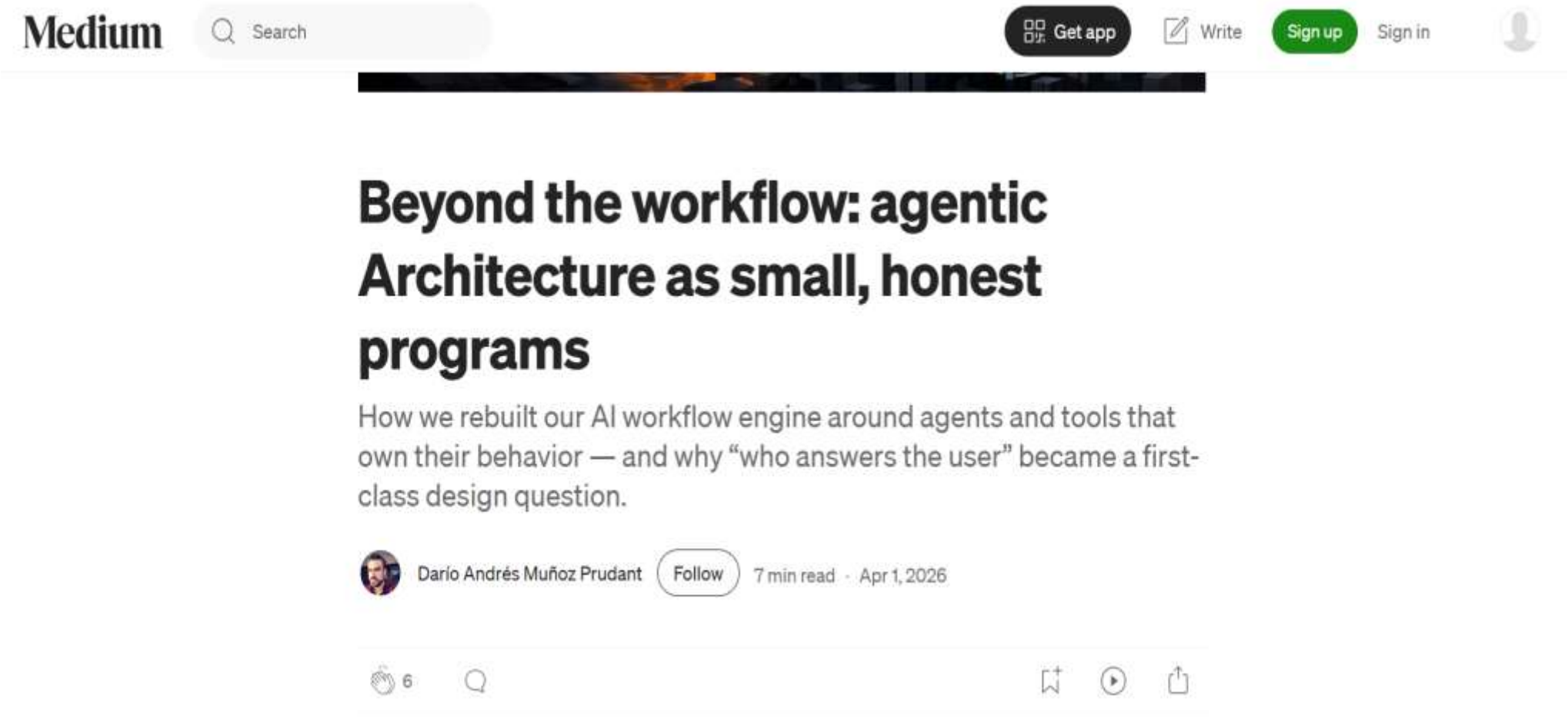
Ingeniería de soluciones con AI

Semestre 2026-1

Docente:

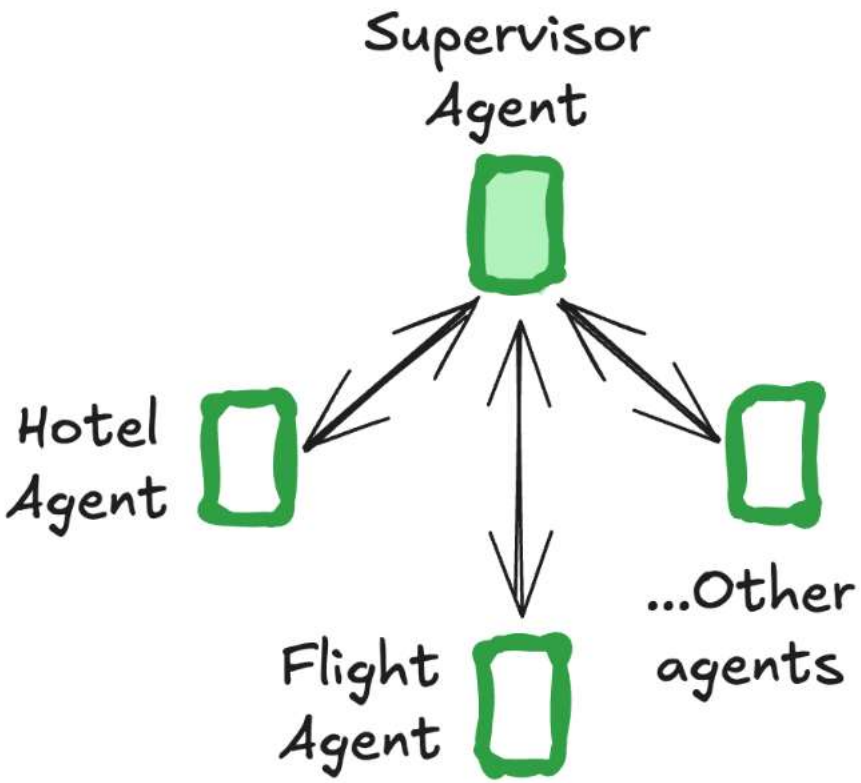
Francisco Macaya

Noticias: *Agentic Arquitecture as small, honest programs.*



Nota: *https://ai.plainenglish.io/beyond-the-workflow-agentic-architecture-as-small-honest-programs-24e6b2facd3a*

Multi – Agents system: *Cuando lo simple no es suficiente.*



"In 2025, the models out there are extremely intelligent. But even the smartest human won't be able to do their job effectively without the context of what they're being asked to do. "Prompt engineering" was coined as a term for the effort needing to write your task in the ideal format for a LLM chatbot. "Context engineering" is the next level of this. It is about doing this automatically in a dynamic system. It takes more nuance and is effectively the #1 job of engineers building AI agents."

Multi – Agents system: 4 patrones para mirar.

Pattern	How it works
Subagents	A main agent coordinates subagents as tools. All routing passes through the main agent, which decides when and how to invoke each subagent.
Handoffs	Behavior changes dynamically based on state. Tool calls update a state variable that triggers routing or configuration changes, switching agents or adjusting the current agent's tools and prompt.
Skills	Specialized prompts and knowledge loaded on-demand. A single agent stays in control while loading context from skills as needed.
Router	A routing step classifies input and directs it to one or more specialized agents. Results are synthesized into a combined response.
Custom workflow	Build bespoke execution flows with LangGraph, mixing deterministic logic and agentic behavior. Embed other patterns as nodes in your workflow.

Nota: *https://docs.langchain.com/oss/python/langchain/multi-agent#router-4*

SubAgents: *Orquestación centralizada.*

¿Cómo funcionan?:

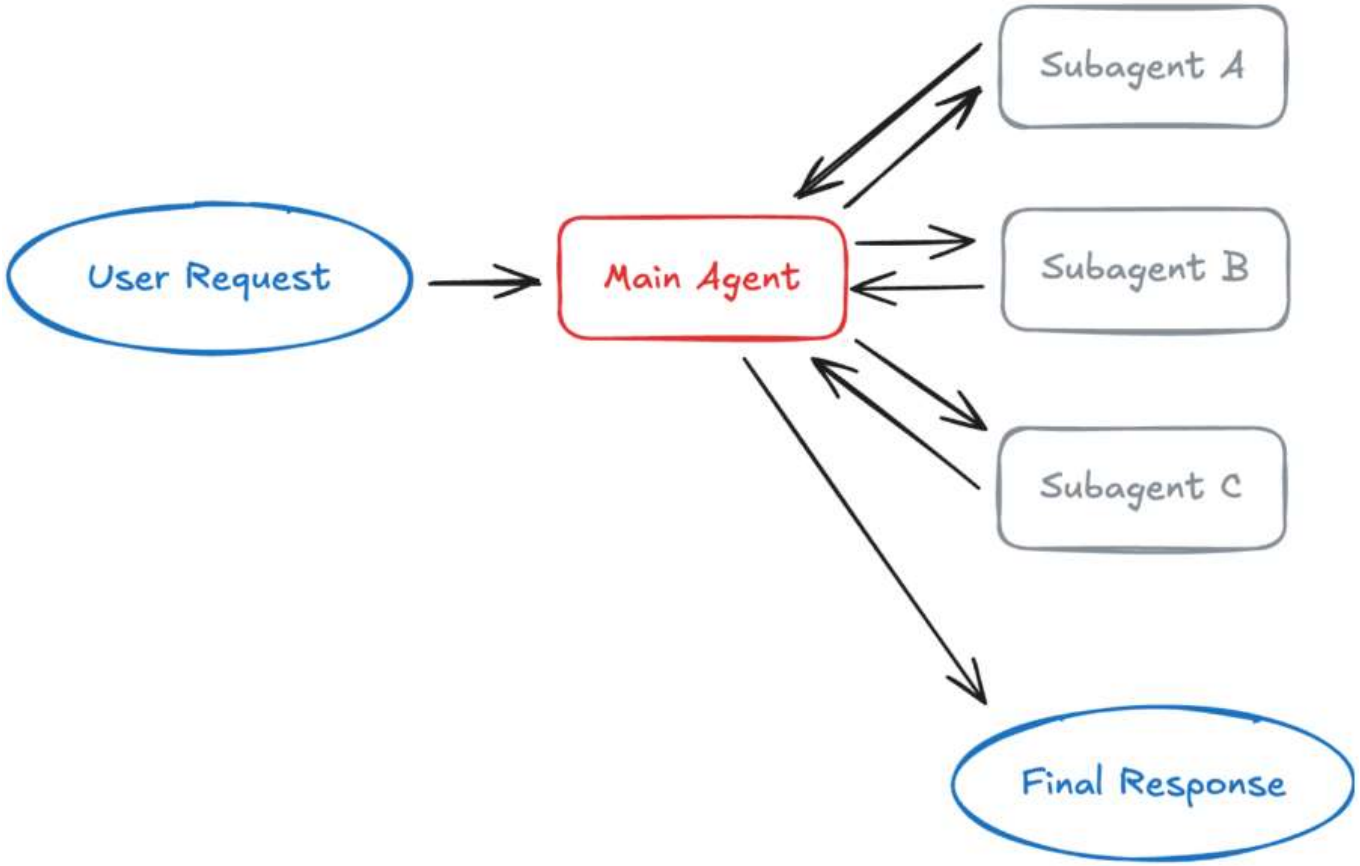
El agente principal decide que subagente invocar, que información proporcional y como combinar los resultados. El agente principal puede invocar a múltiples agentes en paralelo.

Ideal:

Aplicaciones con múltiples dominios distintos donde se necesita un control centralizado del flujo de trabajo y los subagentes no necesitan conversar directamente con los usuarios.

¿Trade-off?:

Añade una llamada al modelo adicional por interacción, ya que los resultados deben volver a pasar por el agente principal.



Nota: *https://docs.langchain.com/oss/python/langchain/multi-agent#router-4*

Skills: *Divulgación progresiva.*

¿Cómo funcionan?:

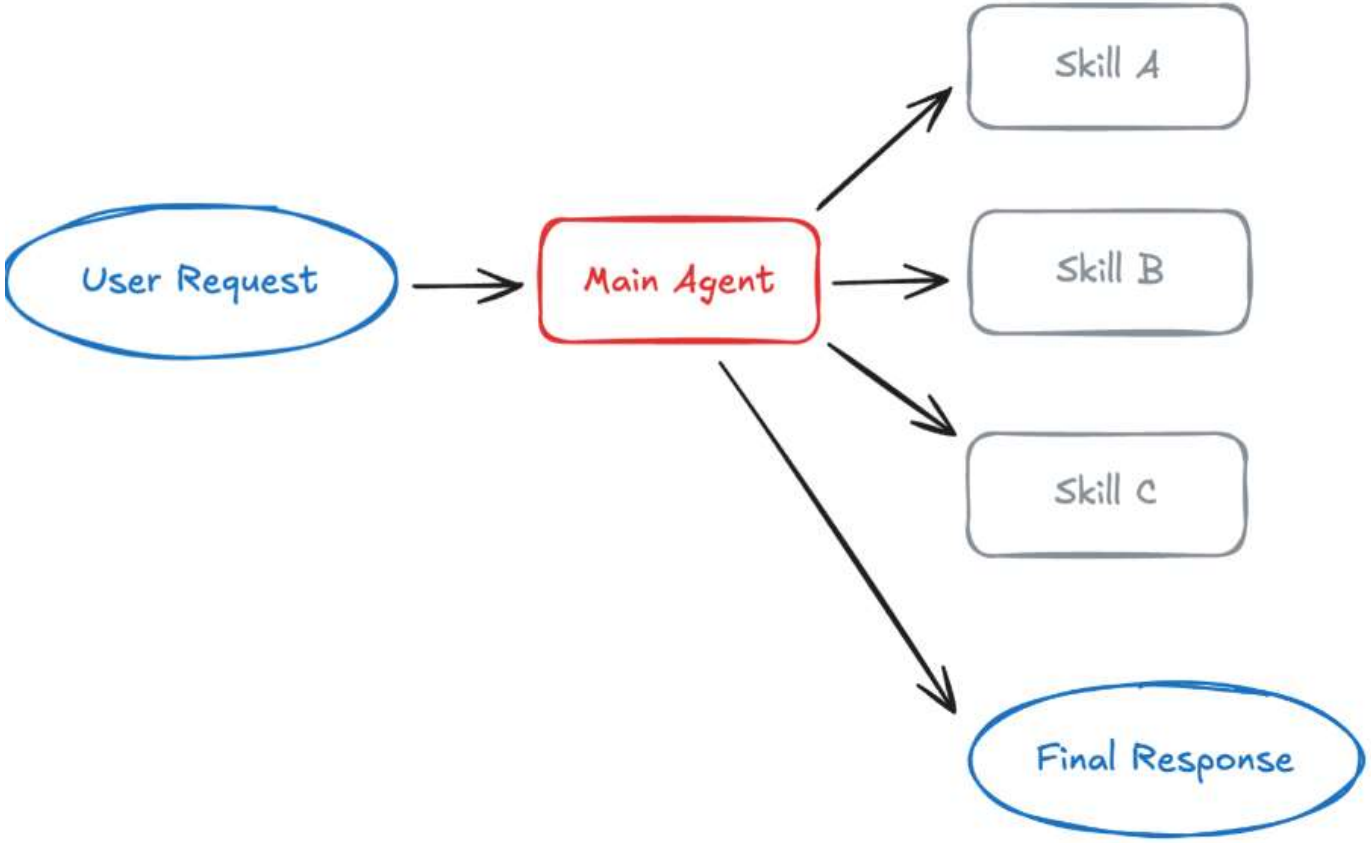
Las habilidades son especializaciones basadas principalmente en prompts, empaquetadas como directorios que contienen instrucciones, scripts y recursos. Al iniciarse, el agente solo conoce los nombres y descripciones de las habilidades. Cuando una habilidad se vuelve relevante, el agente carga su contexto completo.

Ideal:

Agentes únicos con muchas especializaciones posibles, situaciones donde no se necesita imponer restricciones entre capacidades, o equipos distribuidos donde distintos equipos mantienen diferentes habilidades.

¿Trade-off?:

El contexto se acumula en el historial de la conversación a medida que se cargan las habilidades, lo que puede generar una acumulación excesiva de tokens en llamadas posteriores.



Nota: *https://docs.langchain.com/oss/python/langchain/multi-agent#router-4*

Handoffs: *Transiciones impulsadas por el estado.*

¿Cómo funcionan?:

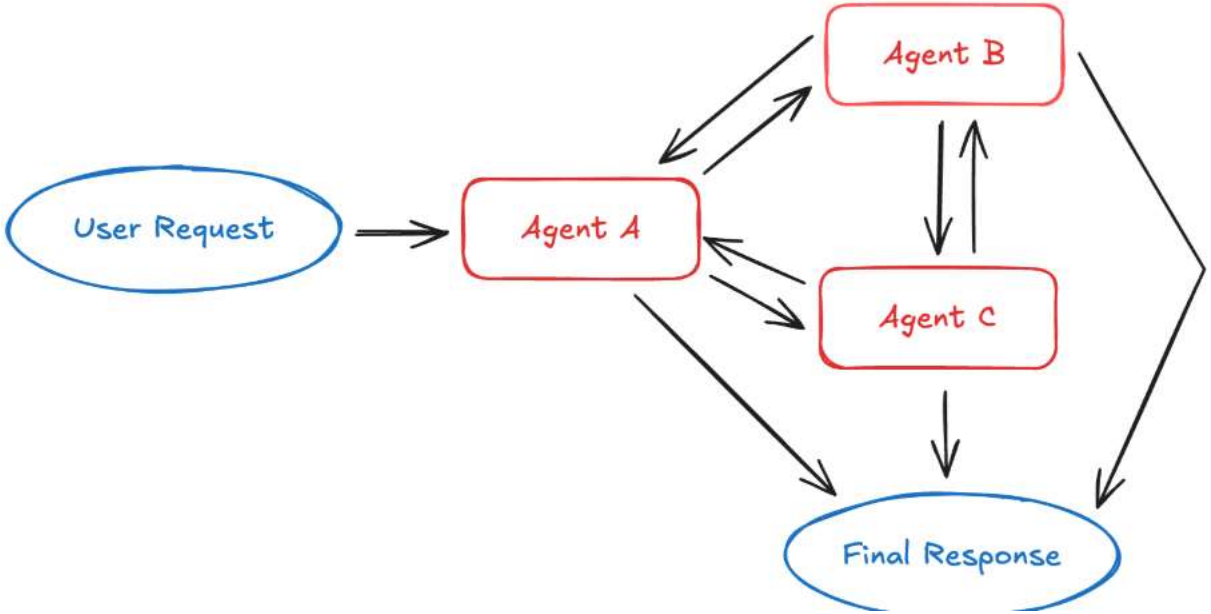
Cuando un agente invoca una herramienta de transferencia, actualiza el estado que determina cuál será el próximo agente en activarse. Esto puede implicar cambiar a un agente diferente o modificar el prompt del sistema y las herramientas disponibles del agente actual.

Ideal:

Flujos de atención al cliente que recopilan información por etapas, experiencias conversacionales en múltiples fases, o cualquier escenario que requiera restricciones secuenciales donde las capacidades se desbloquean solo después de cumplir ciertas condiciones previas.

¿Trade-off?:

Es más dependiente del estado que otros patrones, lo que requiere una gestión cuidadosa del mismo.



Nota: *https://docs.langchain.com/oss/python/langchain/multi-agent#router-4*

Handoffs: *Asistente de un banco.*

Internamente, el agente interactúa y cambia a otros agentes sin que el usuario expresamente haya guiado a esos nodos.

Situación inicial: El usuario escribe "Quiero hacer una transferencia"

El agente activo es el **Agente de Bienvenida**. Su único trabajo es identificar qué quiere el usuario. Cuando detecta que quiere hacer una transferencia, llama a una herramienta llamada `transferir_a_verificacion()`.

¿Qué pasa internamente cuando se llama esa herramienta?

Se actualiza una variable de estado:

```
estado_actual = "verificacion_identidad"
```

Eso dispara un cambio: ahora el sistema activa el **Agente de Verificación**, que tiene un prompt diferente, y solo tiene acceso a herramientas como `verificar_rut()` o `verificar_clave()`. No puede hacer transferencias todavía.

El usuario pasa la verificación. El agente llama a `transferir_a_operaciones()` y el estado cambia:

```
estado_actual = "operaciones_bancarias"
```

Ahora se activa el **Agente de Operaciones**, que sí tiene acceso a `ejecutar_transferencia()`. Esta herramienta no estaba disponible antes porque no se habían cumplido las condiciones previas.

Nota: *https://docs.langchain.com/oss/python/langchain/multi-agent#router-4*

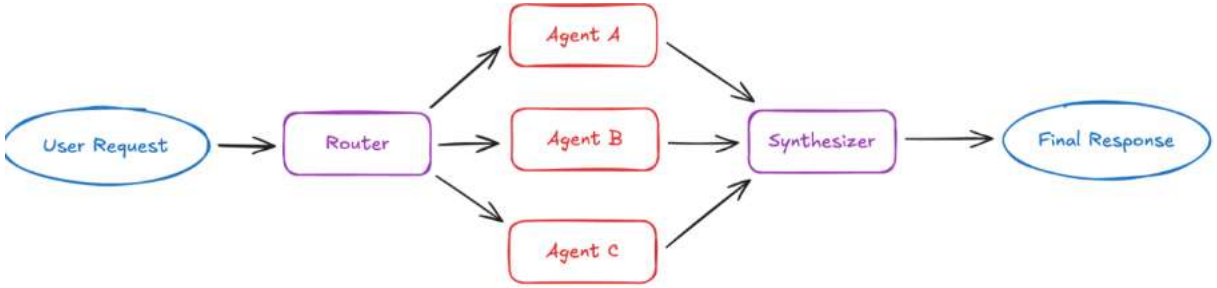
Router: *Dirige y ejecuta en paralelo.*

¿Cómo funcionan?:

El enrutador descompone la consulta, invoca cero o más agentes especializados en paralelo y sintetiza los resultados en una respuesta coherente.

Ideal:

Aplicaciones con verticales distintas (dominios de conocimiento separados), escenarios que requieren consultas a múltiples fuentes en paralelo, o situaciones donde se necesita sintetizar resultados de múltiples agentes.



¿Trade-off?:

El diseño sin estado garantiza un rendimiento consistente por solicitud, pero genera una sobrecarga repetida de enrutamiento si se necesita historial de conversación.

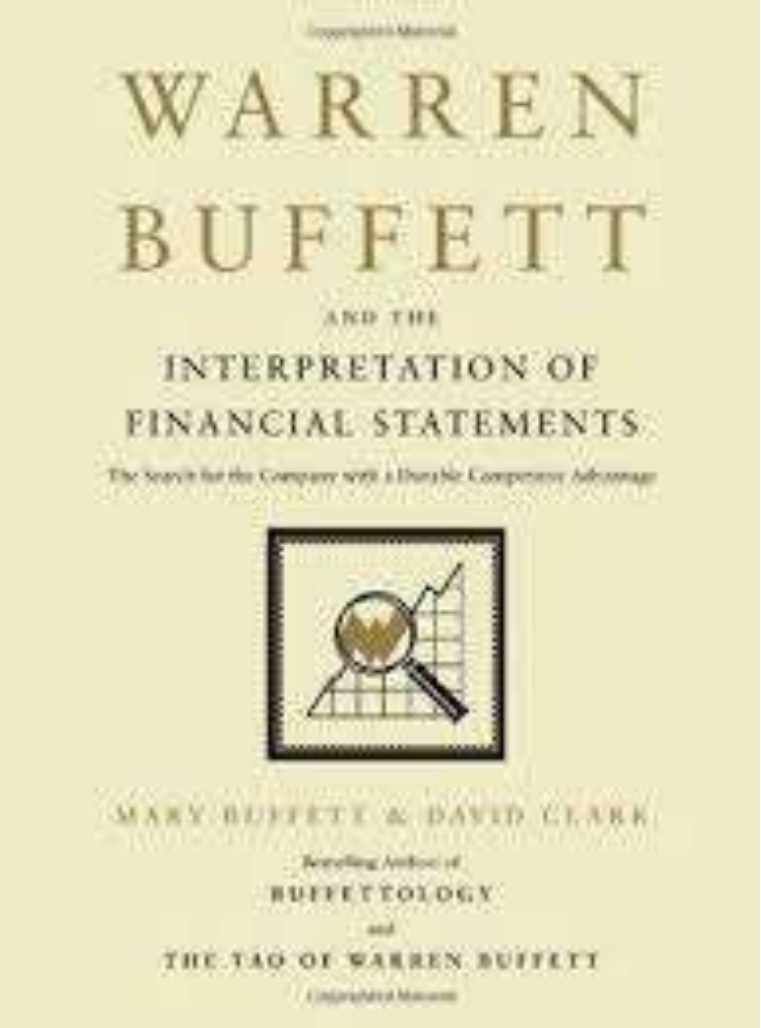
Nota: *https://docs.langchain.com/oss/python/langchain/multi-agent#router-4*

¿Cuál elegir?: *Depende.*

YOUR REQUIREMENTS	PATTERN
Multiple distinct domains (calendar, email, CRM), need parallel execution	Subagents
Single agent with many possible specializations, lightweight composition	Skills
Sequential workflow with state transitions, agent converses with user throughout	Handoffs
Distinct verticals, query multiple sources in parallel and synthesize results	Router

Nota: *https://docs.langchain.com/oss/python/langchain/multi-agent#router-4*

Libro: *Warren Buffet, Interpreation of financial statements.*



Nota: *https://www.buscalibre.cl/libro-warren-buffett-and-the-interpretation-of-financial-statements-the-search-for-the-company-with-a-durable-competitive-advantage/9781416573180/p/2018929?srsltid=AfmBOoqBwiCPD64ZTjIK1B5Vx2dnq53ESfb3gbcHtiHaiNIRIrIOW29H*